DBMS

YOUVA

TRU

NAVNEET



SAFETY LOCK

DBMS

3 Schema Architecture (view of data)

Types of Image classifiers

CNN,

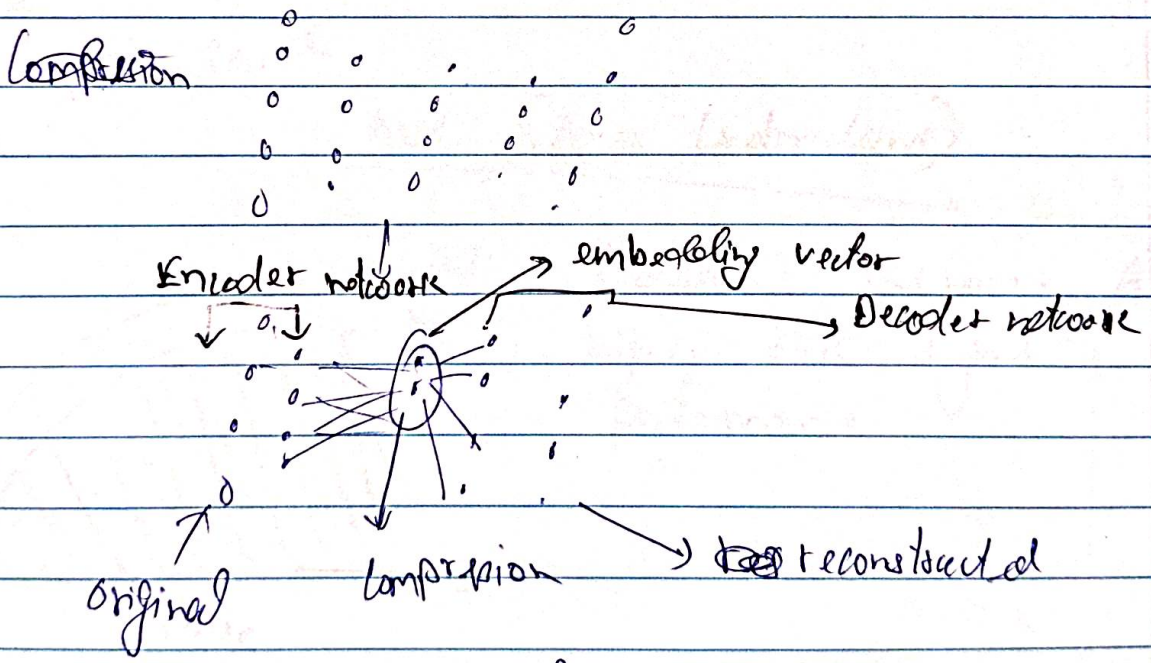
Yolo

ViT → vision transformers

GAN → two networks compete to generate new image that looks real

Autoencoders → used for denoising / Compression
↳ image compression, anomaly detection

unsupervised learning model, that learns about the data



Reconstructed − original = Reconstruction error

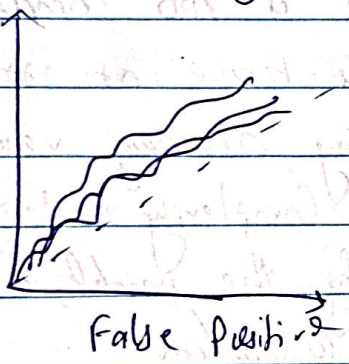
- [x] CNN
- [ ] RAG model
- [ ] How in a PDF the text is taken & read
- [ ] Research paper
- [ ] Yolo model

7358869740 → Airtel ohie

ROC Curve

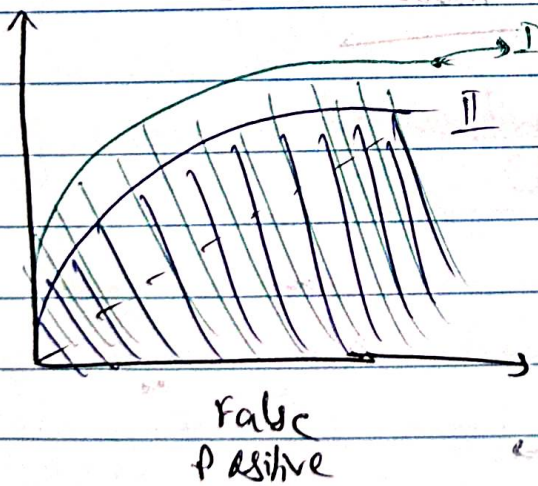
Reciver operating characteristic curve

ability to distinguish btw binary classes (0 & 1)



If the curve is above the diagonal then it shows better performance.

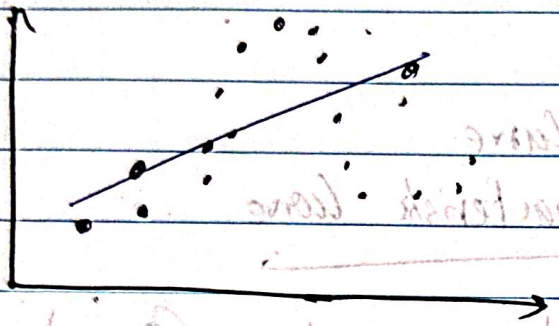
AUC (Area Under Curve)



Predictor I has more AUC higher AUC higher the model performance

Overfitting & Underfitting model

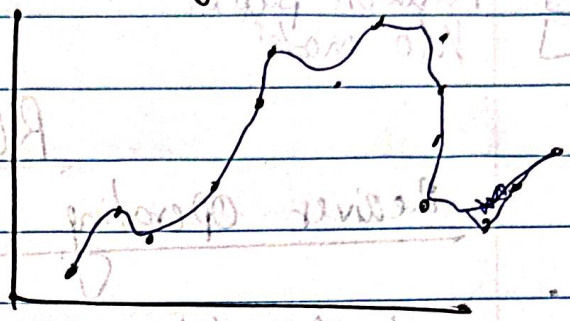
Underfitting model



1) model is to simple & doesn't capture patterns in the data
2) training accuracy low
3) model Complexity low

solutions: use more Complex model & add more features

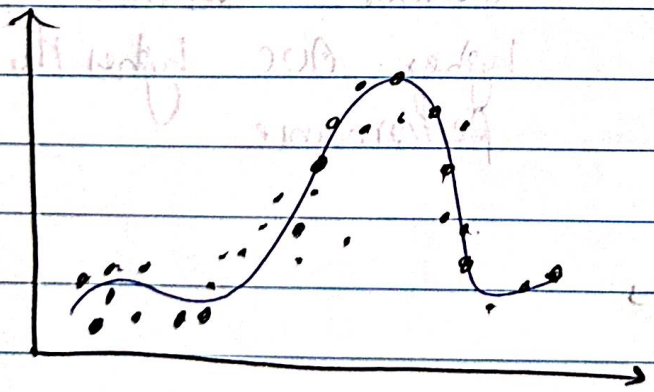
Overfitting model



1) model is too Complex & captures noise or irrelevant patterns
2) training accuracy very high
3) model Complexity high

Simplify the model, use regularization

Best model = Generalized model



CNN

9 9 9 9

To handle variety in digits we need to use simple artificial neural network (ANN)

ReLu is used to bring non linearity in our model

Pooling layers is used to reduce the size

Lenet

Conv → AvgPool → Conv → AvgPool → → → → fully connected layers

Avg pooling ⇒ Sub sampling

optimizers ⇒ SGD → Stochastic gradient descent
↳ adjust the weight by batches.

adam → Combines SGD & RMSprop

RMS prop → root mean sq propagation

loss ⇒ Categorical. means ~~one hot~~ encoded vector

sparse_categorical → 8, or 10, ...

✓ **loss function** → → is the diff btw predicted o/p & actual target value.

→ It guides the model's learning process by showing how well it is performing

→ It helps ~~the para~~ to update the parameters through optimization technique like gradient descent.

Image classification

CIFAR-10 → is a dataset from tensorflow library

60,000 images
10 classes

diff btw softmax & Sigmoid

Softmax → will normalize the probability

Sigmoid → 1 : 0.45
2 : 0.67

Softmax → 1 : $\frac{0.45}{0.45 + 0.67}$ } normalize

GPT → Generative Pre-trained Transformer

LLM

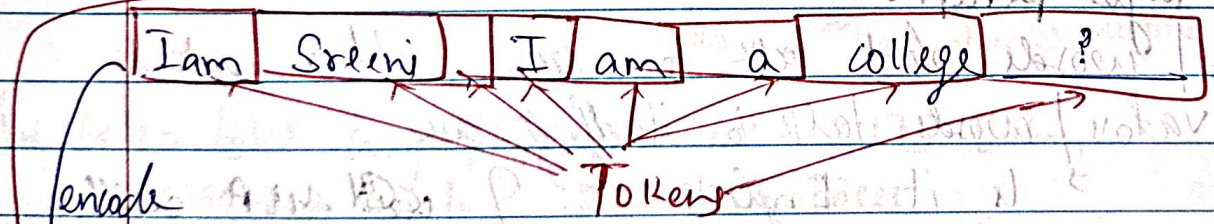
LLM = Data
+
Architecture → is a Neural network → Transformer architecture
↳ helps model to handle sequence of data like sentences or a line of code.
+
Training

Transformers are designed to understand the context of each word in a sentence by considering a relation with every other word

Stochastic Parrot/System → A system is characterized by probability

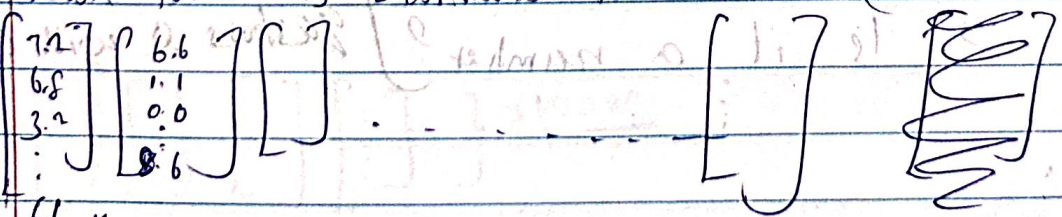
Inside a Transformer

Embedding → Attention → MLP (multi-layer Perception) → Unembedding

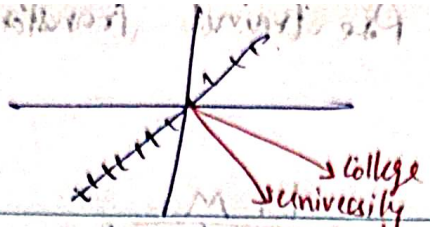


encode

Each token is converted to a vector which contains numbers



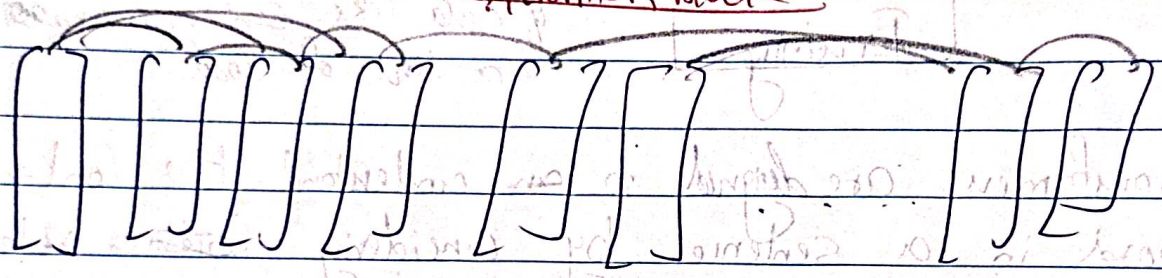
6.6 These vectors in embedding matrix we begin at random & it will change/updated during the training process



→ These vectors are coordinates at very high dimensional space.

→ words with similar meaning like college, university tend to landon vectors that are close to ~~each~~ each other

Attention block



In attention block all the vectors will talk to each other, compare & pass information back & forth to update the values.

A machine learning model | A fashion model

MLP multi layer perception

vectors / words dont talk to each other, instead the vectors / words talk to itself & ask

→ Is it English ?
→ Is it a noun ?
→ Is it a number ?
⋮

} MLP will ask these kind of questions & will update

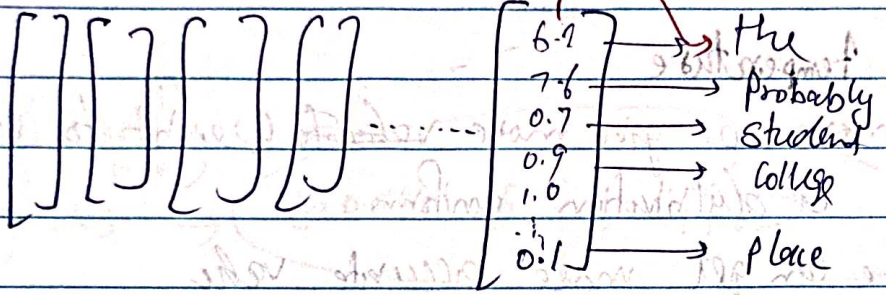
Next word Prediction is Called Logits.



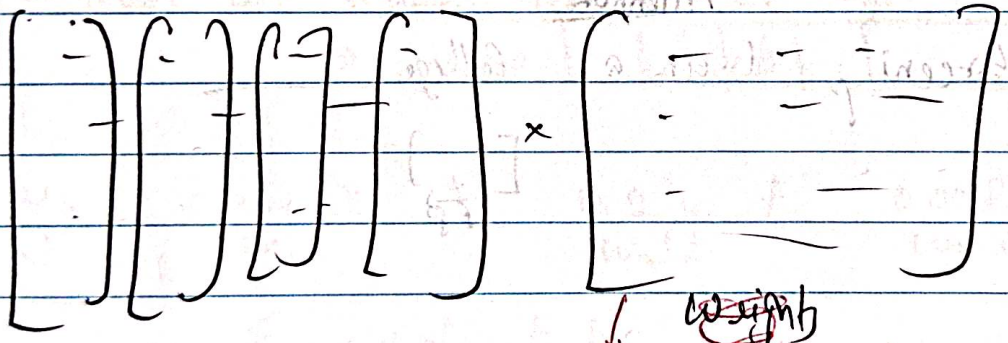
The process repeats with Attention & MLP

repetition many

Attention → MLP → Attention → MLP . . . . . .

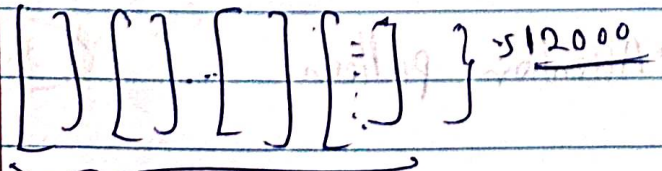


Weights → are the brain that are learned during training & they determine how it behaves



Tunable Parameters ⇒ weights

for GPT3 ⇒ total weights/hyperparameter = 175 billion
organised into 27,000 matrices
12,288 dimension



$\frac{e^{x/T}}{\sum e^{x/T}}$

Softmax

x^1 → [0.8, 0.5, 3.4, 3.9, -2.3] → Softmax → [0.04, 0.01, 0.08, 0.09, 0.03]

we can add temperature

when T is more we give more ~~value~~ weight to smaller value to make the probability uniform

if $T=0$, we can get more accurate value

if $T>0$ eg $T=0.2$ it might not be accurate.

T cant be >2

Attention

I	am	Sreeni	,	I	am	a	College

E_1 E_2 ... E_8

W_Q

Q_1 (whether any adj are there or not)

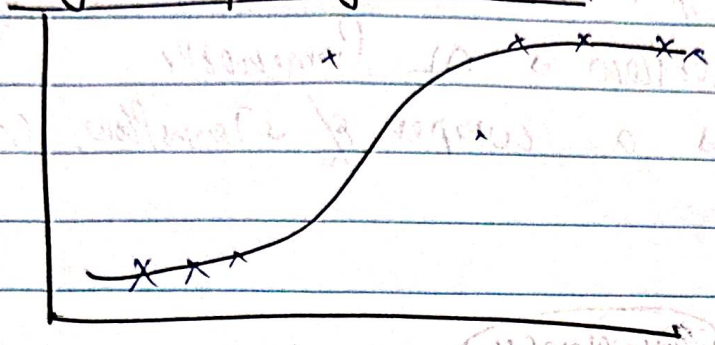
K_1 key vector

this will have the more value, so it is suitable to match with the new word

→ Attention pattern

Tunable parameters in Attention → Query, Key & Value

Sigmoid / Logit function



Neuron →

Age = 43 → [$y = 0.042x - 1.53$ | $z = \frac{1}{1+e^{-y}}$] → 0.57

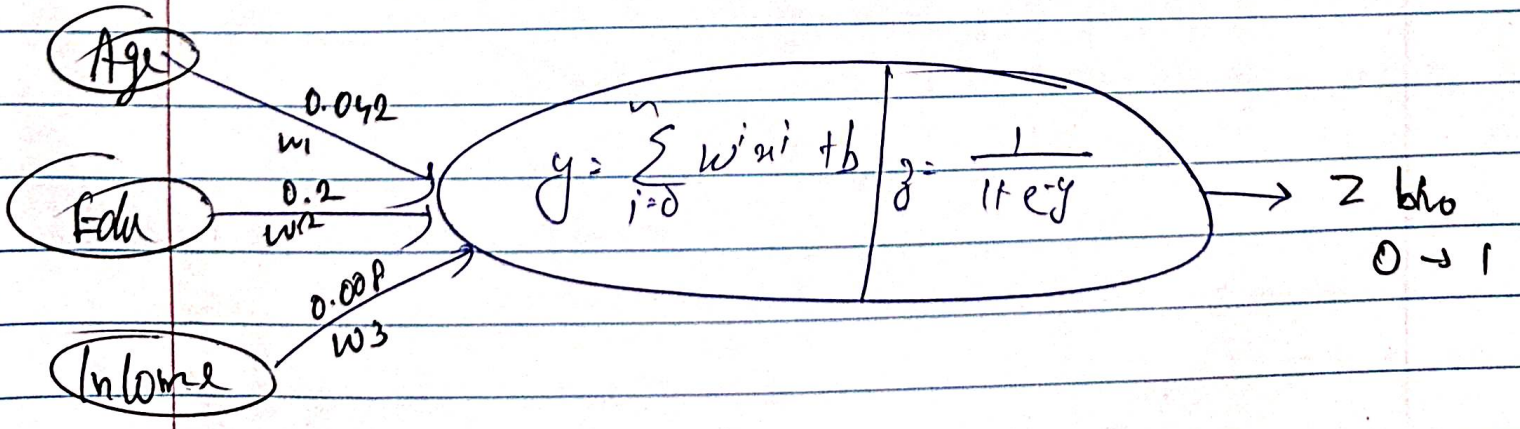
if value > 0.5 will buy insurance

linear eq — Activation fn (Sigmoid fn)

If there are multiple features like age → income, & education

$$y = \underset{w_1}{0.042x} + \underset{w_2}{0.2y} + \underset{w_3}{0.008z} - 1.53$$ (bias)

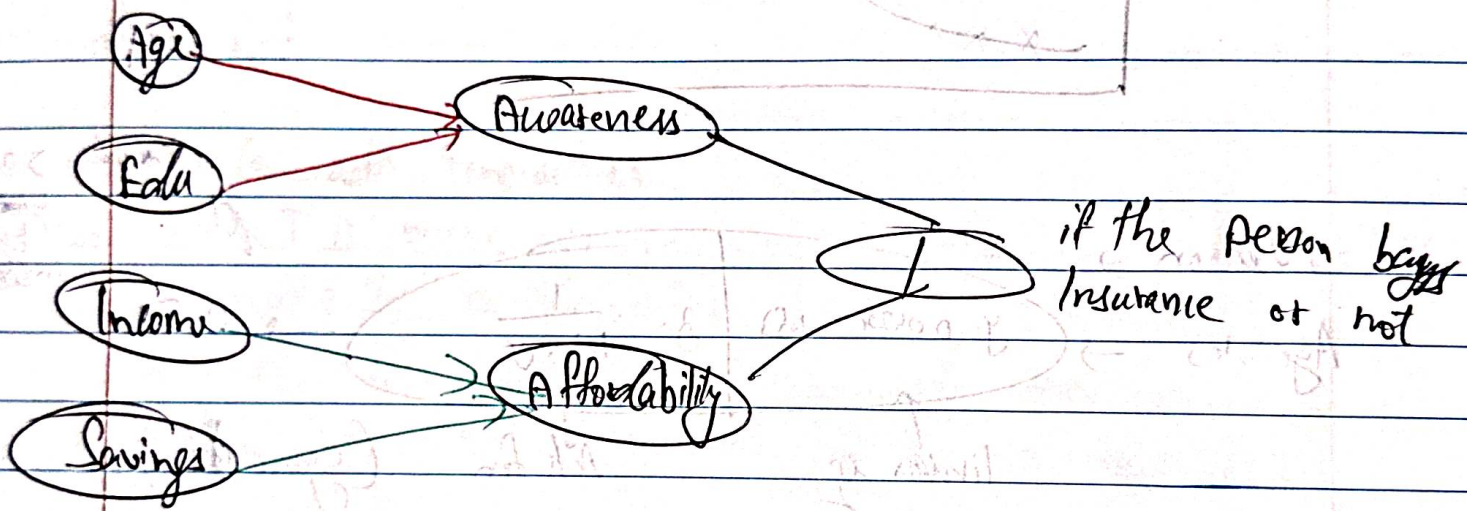
$$= \sum_{i=0}^{n} w_i x_i + b$$



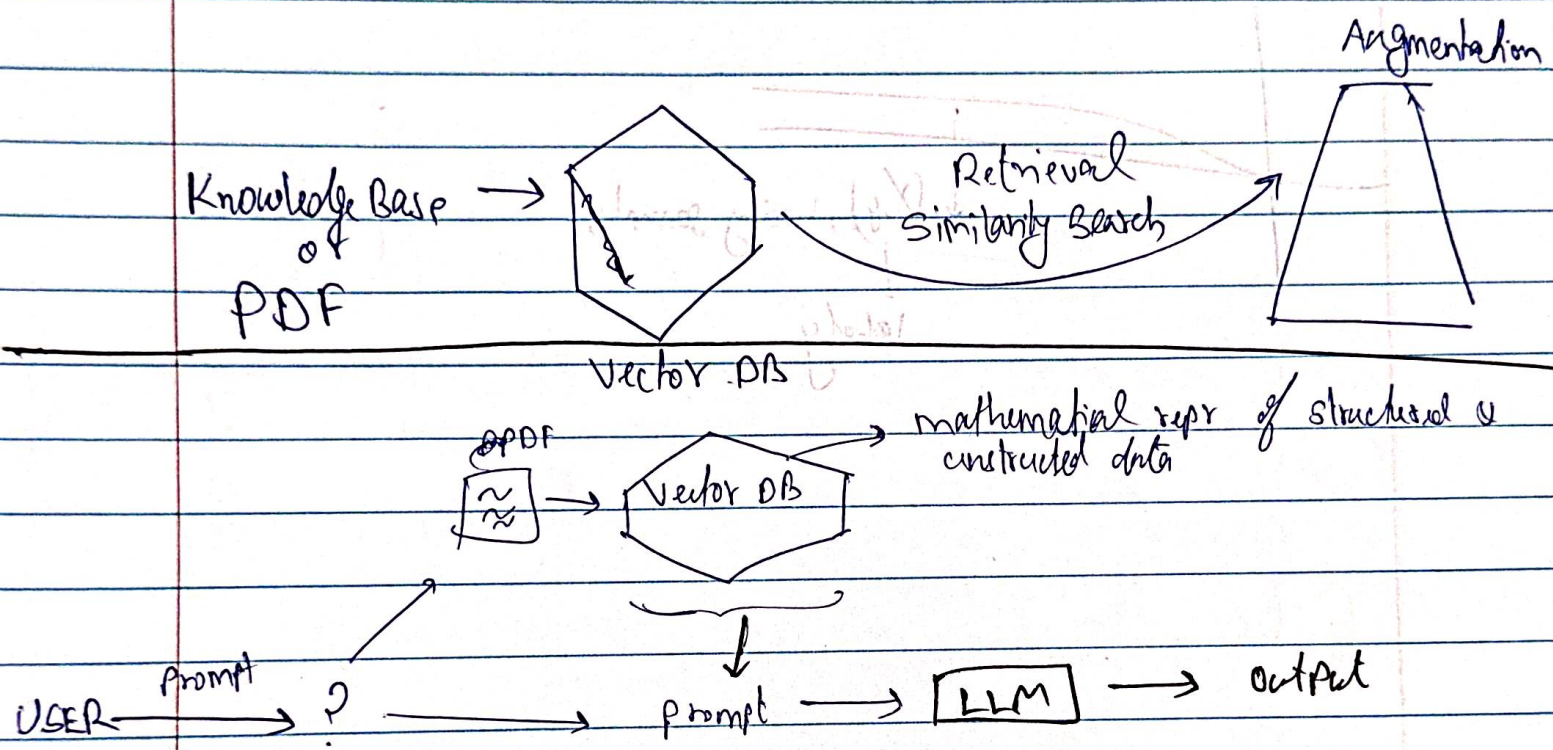
facebook ↑ Google ↑

Pytorch & Tensorflow → DL Frameworks

Keras is a wraper of Tensorflow, CNTK & Theano



RAG


Augmentation
Knowledge Base of PDF
Retrieval Similarity Search
Vector DB
PDF
Vector DB
mathematical repr of structured & unstructured data
USER
prompt
?
prompt
LLM
output

deeplab v3+ for feature extraction
RA-CNN for classification

150 epochs ↑ 32 batch size ↑

layers batch size reduce models ability to generalize
ie : updates to weights are less freq.

SPPF → spatial pyramid pooling fast
↳ generate fixed feature representation of obj of various size without resizing the img or reducing spatial info loss

Yolo v8

Backbone, neck & head

Backbone → feature extractor

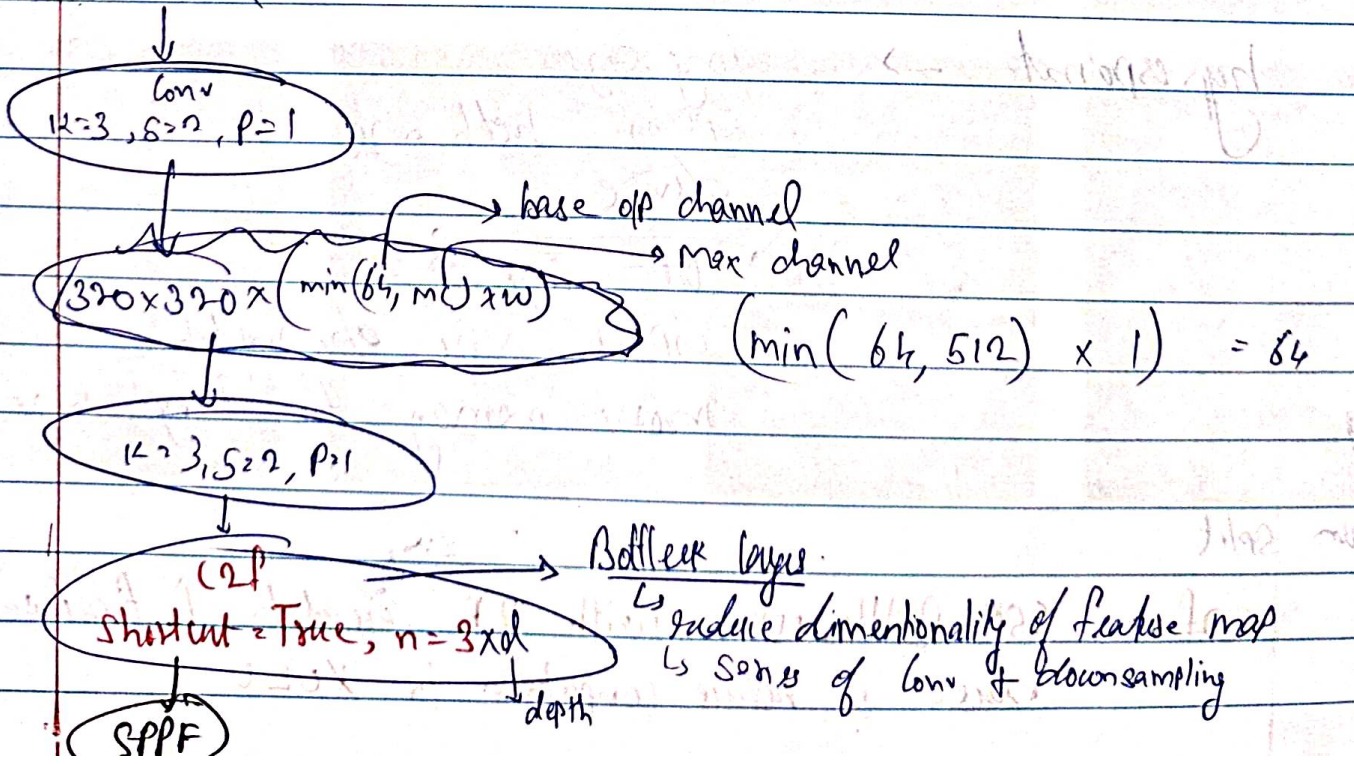
neck → combines the feature from various layers

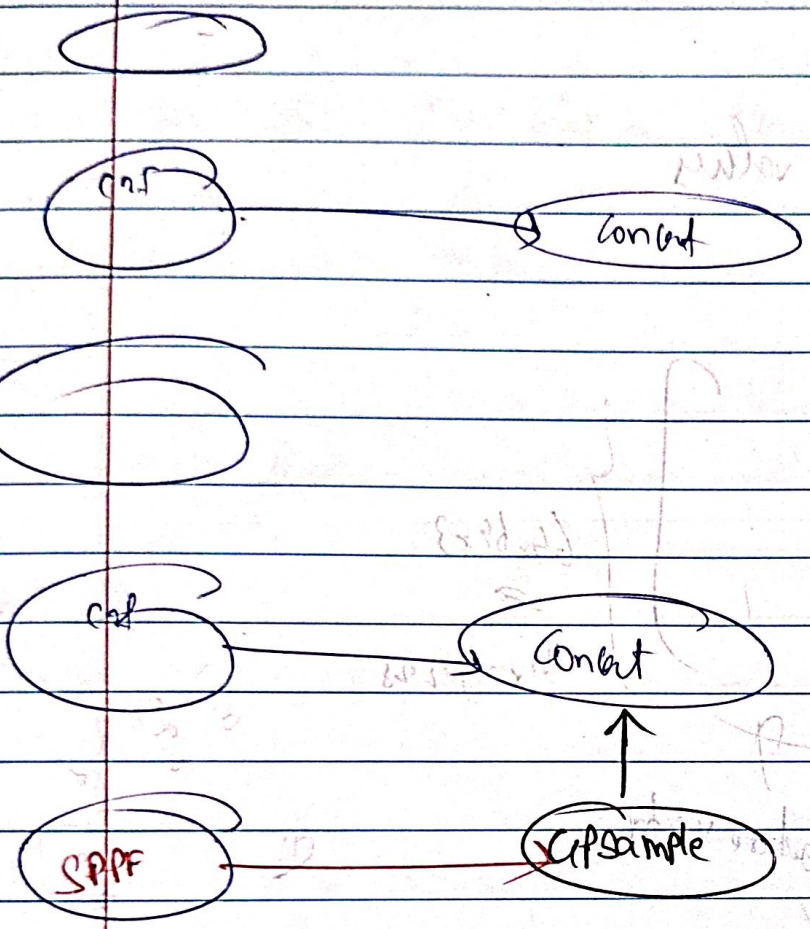
Head → predicts the classes & bounding box

parameters → depth multiple, width multiple, max channels
determines how many bottleneck blocks are in C2f block

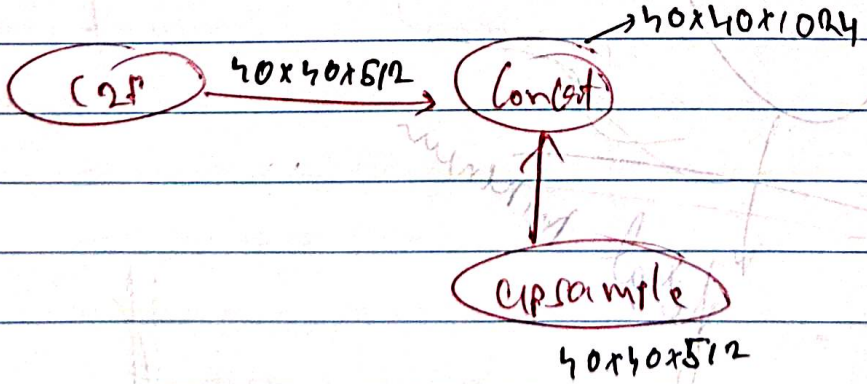
spatial resolution → determines the o/p

640 × 640 × 3 → channel





upsample layer → ↑ the feature map resolution of SPPF



CNN

MLP

Convolutional layers → Pooling layers → Flatten → [⋮] → MLP → output

---

Neuron → A thing that holds a number btw 0 → 1



-0.2 → low activation

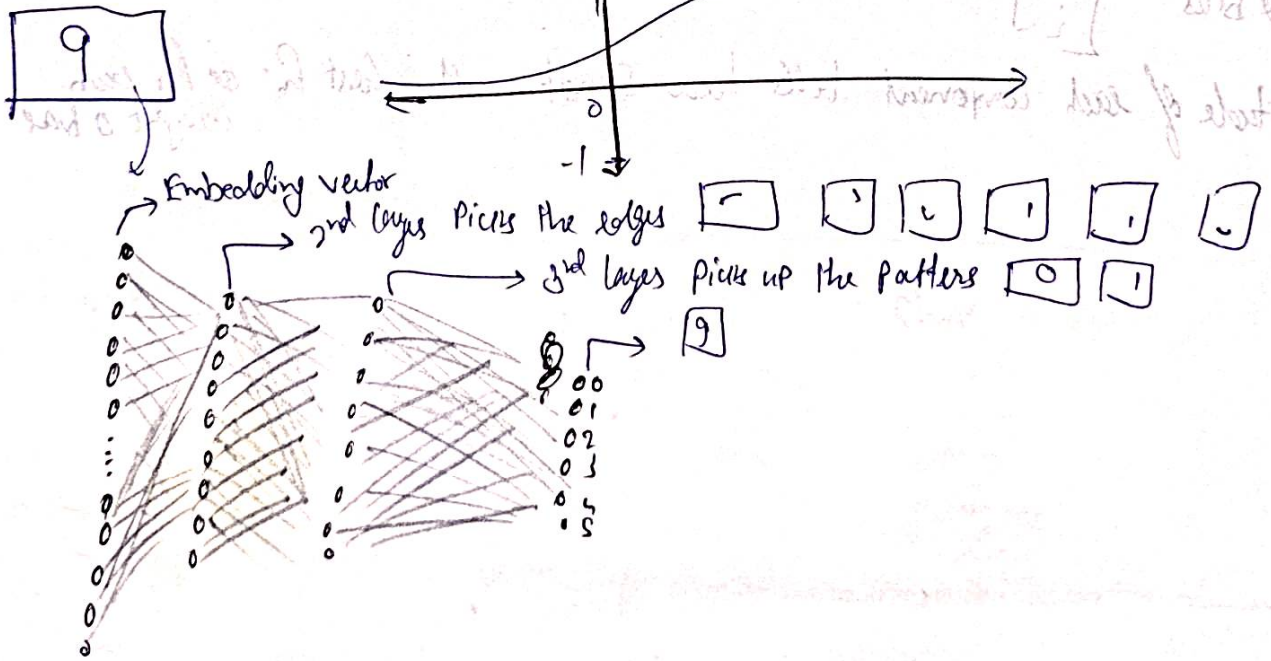


→ 1 high activation

Activation → Acc to Brightness of a digit the activation no is

Activation for each neuron is based on the weighted sum of all activations in the prev layers + bias

Sigmoid fun $\sigma(x) = \frac{1}{1+e^{-x}}$



bias → is an indication ~~whether~~ that shows a neuron is active or inactive

normal parameters → weight & bias

Back Propagation.

Cost → of a single training is
we need to take the o/p the network gives
& the o/p that we want to give.
we need to add up the SQ of diff b/w each comp

$$\begin{cases} (0.43 - 0.00)^2 + \\ (0.25 - 0.00)^2 + \end{cases}$$

Adding all of them & averaging will give the total cost of the network

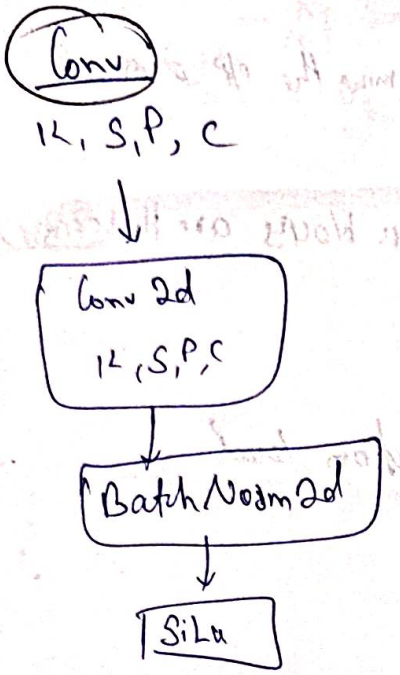
negative gradient of cost fn tells how to change the weight & bias

Back Propagation → is the algorithm for computing the gradient

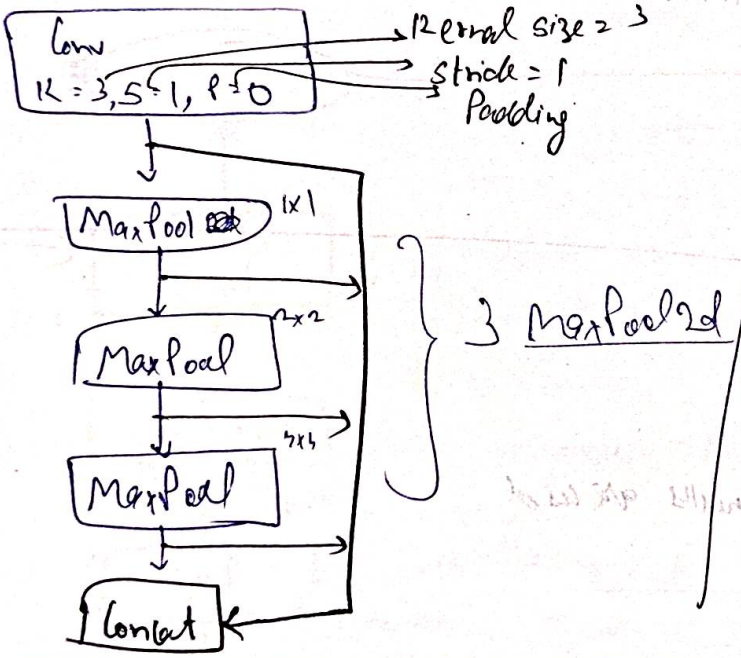
weight & bias $\begin{bmatrix} \vdots \\ i \end{bmatrix}$

The magnitude of each component tells how sensitive the cost fn is for each weight & bias

Yolo V5



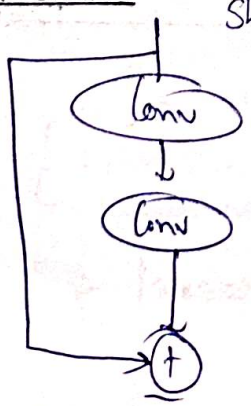
SPPF → Spatial Pyramid Pooling Fast

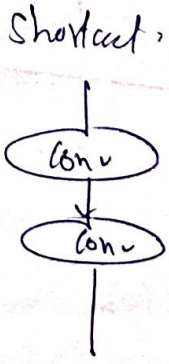


SPPF

↳ ① handle i/p img of diff size without requiring a fixed i/p size.

Bottleneck





Bottleneck is similar to Resnet.
Bottleneck has shortcut, where as ResNet dont have

Yolo v8 Parameters → d (depth_multiple) → determine how many bottleneck blocks are there in c2f block
w (width_multiple) } → determine the o/p channel
mc (max_channel) }

Backbone
↳ extract feature at different resolution level

640x640x3

Conv
k=3, s=2, p=1

320x320x(min(64, mc)xw)
= 64
base o/p channel
max channel
width_multiple Parameters

Conv
k=3, s=2, p=1

160x160x128

C2f
Shortcut = True
n = 3xd
depth_multiple
determines how many bottlenecks are used

Conv
k=3, s=2, p=1

80x80x256

C2f

Conv

C2f

20x20x1024

SPPF

Bottleneck layers → to reduce the no. of parameters & computational complexity by compressing the i/p with 1x1 Conv.
by preserving impt features

SPPF

C2f

Detect

Concat

Upsample

C2f

Detect

Concat

Upsample

40x40x1024

20x20x1024

Concat

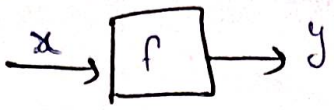
C2f

Detect

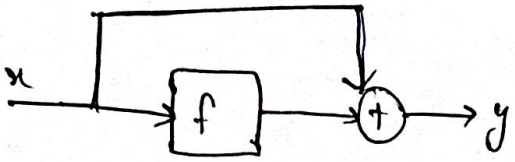
Residual Network

ResNet 50 → is a pretrained model which contains 50 layers → 50 residual blocks so in each residual block there are atleast 1-2 CNN layers.

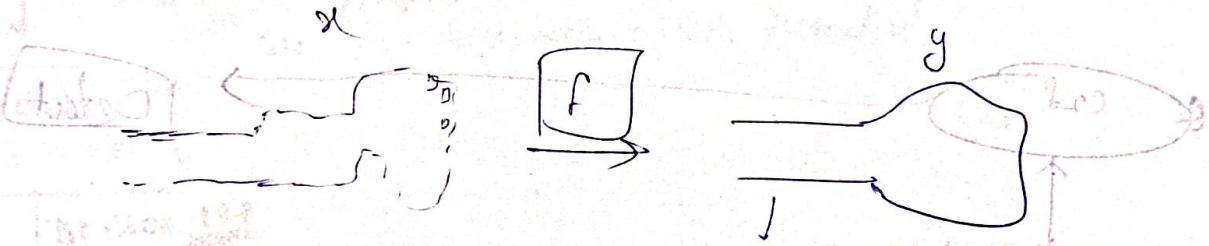
residual fxn.



$y = f(x)$



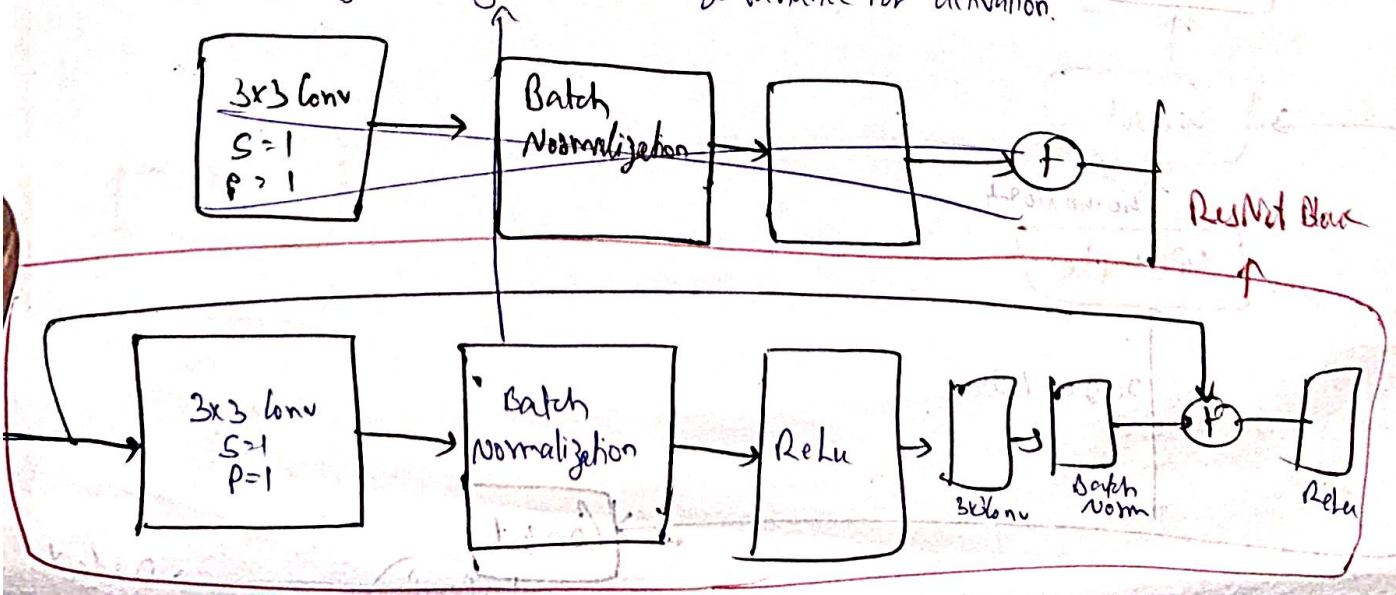
$y = x + f(x)$



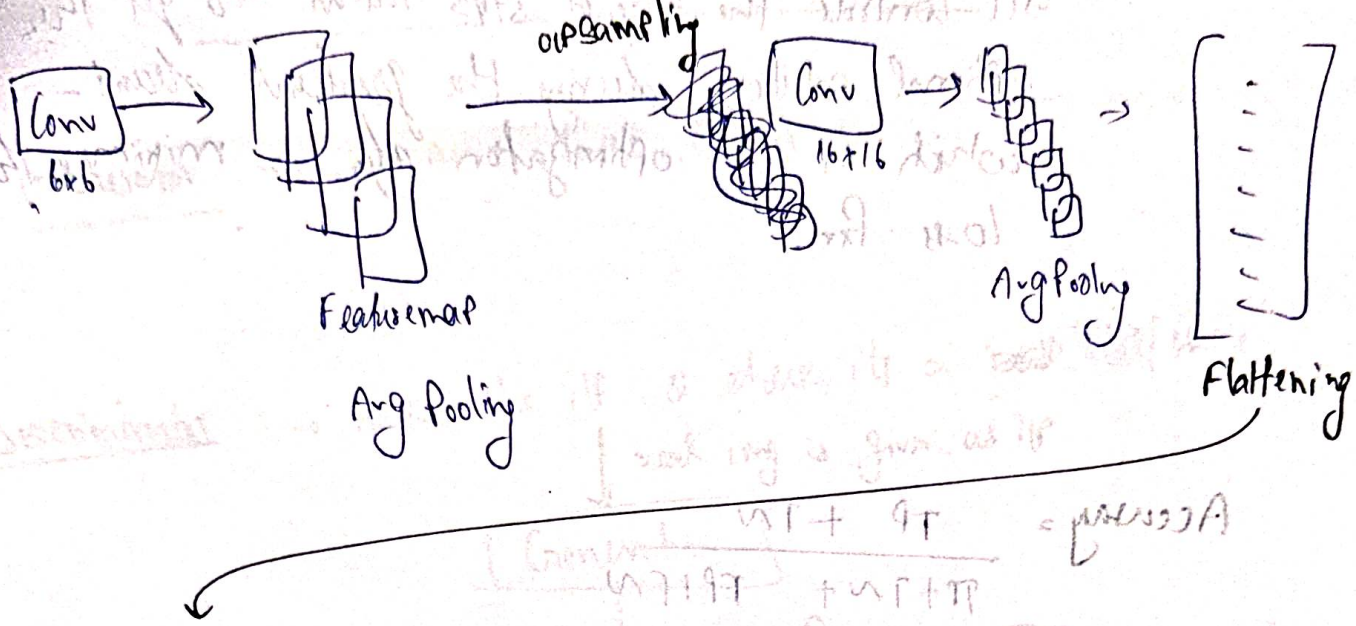
most of the info in the o/p y actually exist in the low resolution x.

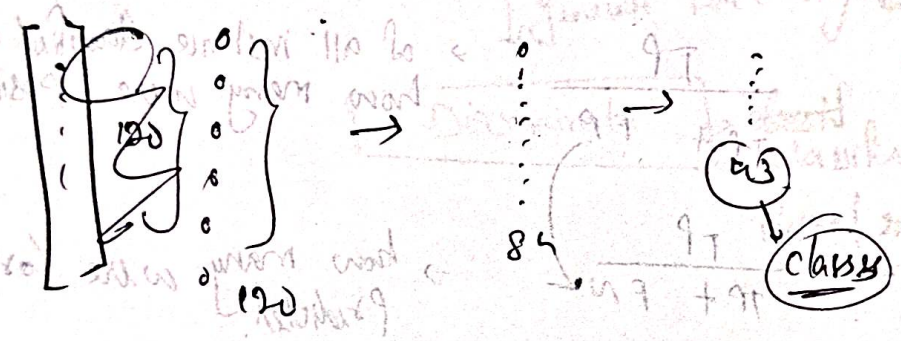
That little features called residue is captured by the function or residual block.

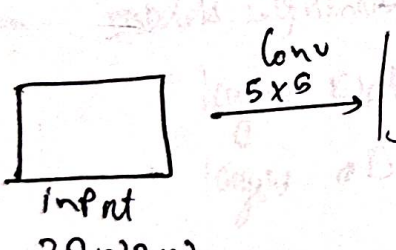
Batch Norm → normalize the i/p of each layer to stabilize & accelerate training by adjusting the mean & variance for activation.



LeNet Architecture







⇒ 32-5 ⇒ 27
27+1 = 28

⇒ (input - filter + 1) / stride

lr learning rate → tells us how fast the model being trained by adjusting the weights.

→ It controls the size of steps taken to get the optimal solution during the gradient descent — which is an optimization algo to minimize the loss fun.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$ → of all instance classified as +ve how many were positive

$$\text{Recall} = \frac{TP}{TP + FN}$$ → how many were correctly predicted

$$\text{F1 Score} = 2 \times \frac{(\text{Prec} \times \text{Recall})}{\text{Prec} + \text{Recall}}$$

---

cnn.

CL

layers.Conv2D(filters=32, kernel_size=(3,3), activation='relu', input_shape=(32,32,3)),

layers.MaxPooling2D((2,2)),

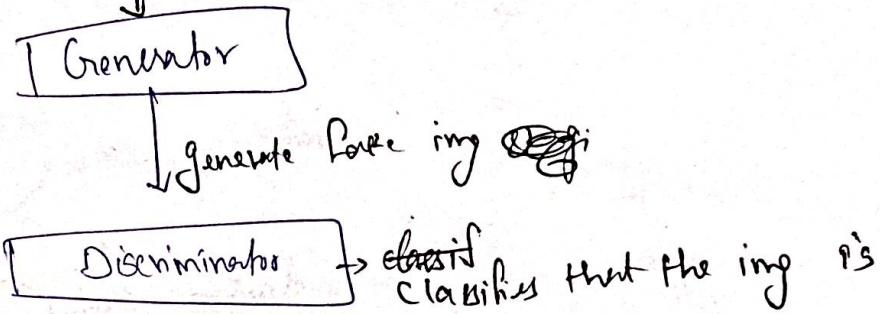
layers.Dense(64, activation='relu')
layers.Dense(10, activation='softmax')

GAN

Generative Adversarial Network

Generator → make fake inputs

Discriminator → take the i/p & check it's a real/fake



```
ann = models.Sequential([
        layers.Flatten(input_shape=(32,32,3)),
        layers.Dense(3000, activation='relu'),
                    ,1000
                    ,10          sigmoid)
    ])
ann.compile(optimizer='SGD',
            loss='sparse_categorial_cross entropy',

ann.fit(x_train, y_train, epochs=5)
```